KGP Talkie
LEARN. BUILD. TALK AI

BEGINNER FRIENDLY

Hugging Face Introduction

Step-by-step study material for using models, datasets, Spaces, pipelines, tokenizers, and AutoClasses.

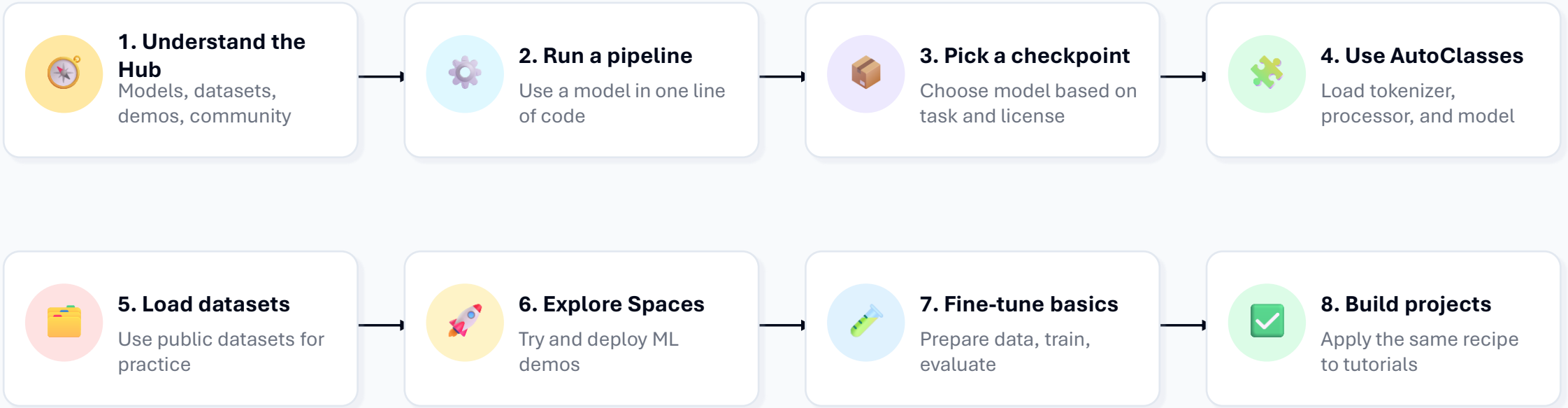
🤗 Start Here

Updated and expanded from your original 12-slide PDF.

KGP Talkie
LEARN. BUILD. TALK AI

Learning path for beginners

Follow this order while teaching or studying. Each step builds on the previous one.



Study rule: do not memorize every class. First understand the pattern: task → checkpoint → tokenizer/processor → model → output.

What is Hugging Face?

Think of it as GitHub + app store + documentation hub for machine learning.

Community platform

Browse, share, and use AI models built by the community.

Model Hub

Find pretrained models for text, image, audio, and multimodal tasks.

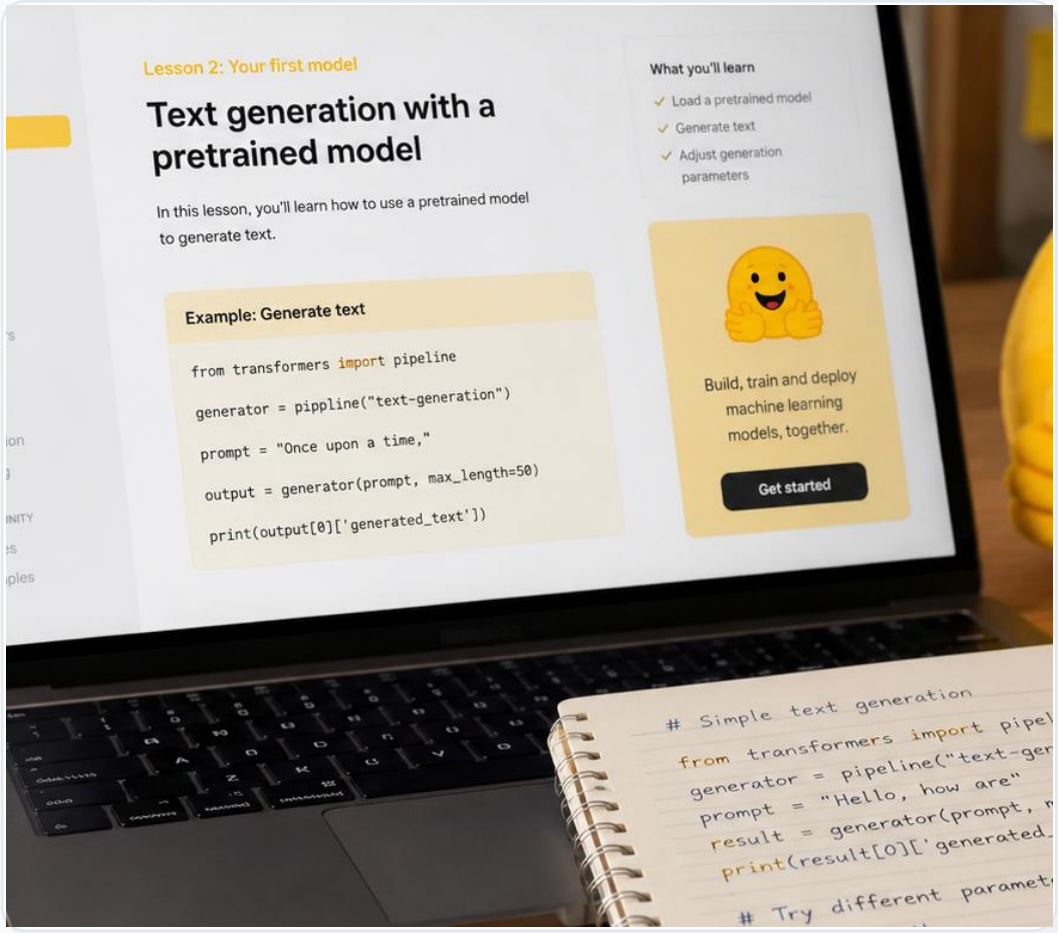
Datasets

Explore datasets with dataset cards and examples.

Spaces

Try machine learning demos in the browser.

Public homepage says: Browse 2M+ models



The Hugging Face ecosystem

KGP Talkie
LEARN. BUILD. TALK AI

You will mostly use these four pieces in beginner tutorials.

🤗 Hub

Model repos, dataset repos, Spaces repos, files, README, versions

⚙️ Transformers

Python library to load tokenizers, processors, models, and pipelines

🗂️ Datasets

Library and Hub pages to search, inspect, and load datasets

🚀 Spaces

Deploy demos using Gradio, Streamlit, Docker, or static apps

KGP Talkie
LEARN. BUILD. TALK AI

Setup before running tutorials

Keep installation simple first. Do advanced GPU setup later.

1. **Create environment**

   Use conda or venv so your project stays clean.

2. **Install libraries**

   Start with transformers, datasets, accelerate, and torch.

3. **Run a pipeline**

   Verify the setup before writing advanced code.

4. **Login only when needed**

   Use a token for gated models or pushing files to Hub.

```
# Option 1: create a clean Python environment
python -m venv hf
# Windows: hf\Scripts\activate
# macOS/Linux: source hf/bin/activate

pip install -U transformers datasets accelerate torch

# Optional login for gated models / uploads
huggingface-cli login
```

Beginner advice: first run everything on CPU. Once the logic is clear, then move to GPU or Colab.

Transformers library: the practical idea

Transformers gives you reusable building blocks for pretrained models.

KGP Talkie
LEARN. BUILD. TALK AI

Why beginners love it

- One common API for many model types
- Works with text, image, audio, and multimodal tasks
- Gives ready-made pipelines for quick inference
- Lets you fine-tune models later using Trainer

Use it with

PyTorch

TensorFlow

JAX

Pretrained model → **Your task** → **Useful output**

KGP Talkie
LEARN. BUILD. TALK AI

What is a pipeline?

A pipeline hides the boring steps and lets you focus on the task.

Input → Pre-processing → Model inference → Post-processing → Output

Hands-on 1: first inference pipeline

Goal: run a pretrained model without understanding every internal detail.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love learning Hugging Face!")
print(result)

# Example output:
# [{'label': 'POSITIVE', 'score': 0.99}]
```

What happened?

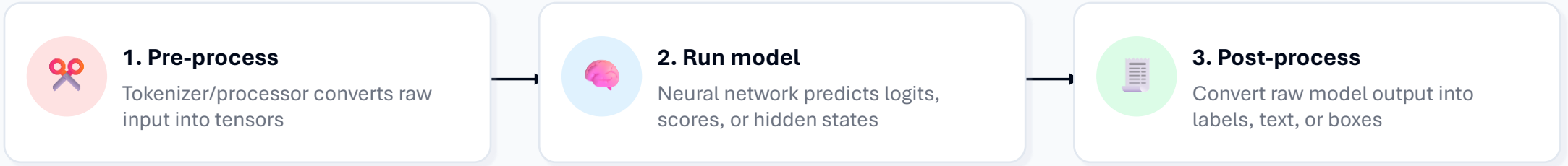
1. Pipeline downloaded a default sentiment model.
2. It converted text into tokens.
3. The model predicted a label.
4. Output became human readable.

Your first Hugging Face win: one task, one line

KGP Talkie
LEARN. BUILD. TALK AI

Behind the pipeline

The same idea appears again and again in Hugging Face tutorials.



Beginner mental model

Whenever you see pipeline(), remember: input → conversion → model → readable output.

KGP Talkie
LEARN. BUILD. TALK AI

Common pipeline tasks

Start with simple text tasks, then move to image and audio tasks.

Text Classification
Sentiment, spam, topic

Question Answering
Context + question → answer

Fill-Mask
Predict missing word/token

Summarization
Long text → short summary

Translation
One language → another

Speech Recognition
Audio → text

Image Classification
Image → class label

Object Detection
Image → boxes + labels

When confused, search by task first. Then select a checkpoint trained for that task.

Model checkpoints: what are you downloading?

A checkpoint is a saved model state: architecture + learned weights + config files.

Checkpoint = saved model package

It usually contains:
- config.json
- model weights
- tokenizer files
- model card README
- license and usage notes

Model family	Dataset / Task	Checkpoint name
BERT	Wikipedia + Books	bert-base-uncased
DistilBERT	SQuAD QA	distilbert-base-cased-distilled-squad
ViT	ImageNet	google/vit-base-patch16-224

Tip: checkpoint name usually tells you model + size + training data/task

KGP Talkie
LEARN. BUILD. TALK AI

Read the model card before using a model

A model card is the README of a model repo. It protects you from wrong choices.

Task

Is it trained for your use case?

License

Can you use it in your project or course?

Evaluation

What metrics and benchmark data are shown?

Limitations

What should you not use it for?

Example code

Can you reproduce the sample quickly?

Framework

Does it support Transformers / PyTorch / etc.?

Beginner habit: never copy a checkpoint name blindly. Read the card first.

KGP Talkie
LEARN. BUILD. TALK AI

Tokenizer: text becomes numbers

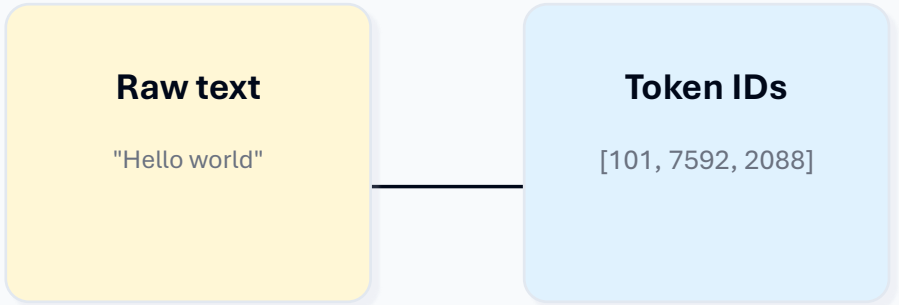
Models do not understand raw text directly. They work with token IDs.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-
uncased")

text = "In a hole in the ground there lived a hobbit."
print(tokenizer(text))

# output contains input_ids, token_type_ids, attention_mask
```



AutoTokenizer loads the correct tokenizer for the checkpoint

AutoClasses: beginner-safe loading pattern

KGP Talkie
LEARN. BUILD. TALK AI

AutoClasses infer the correct architecture from the checkpoint.

`AutoTokenizer`	Text tokenization
`AutoImageProcessor`	Image preprocessing
`AutoProcessor`	Multimodal / layout inputs
`AutoModelForSequenceClassification`	Text classification model
`AutoModelForTokenClassification`	NER / token tagging

```
from transformers import AutoTokenizer,
AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model =
AutoModelForSequenceClassification.from_pretrained(checkpoint)

# Same pattern works for many checkpoints.
```

Beginner rule: pipeline() is fastest for inference. AutoClasses are better when you want control.

Datasets: discover, inspect, load

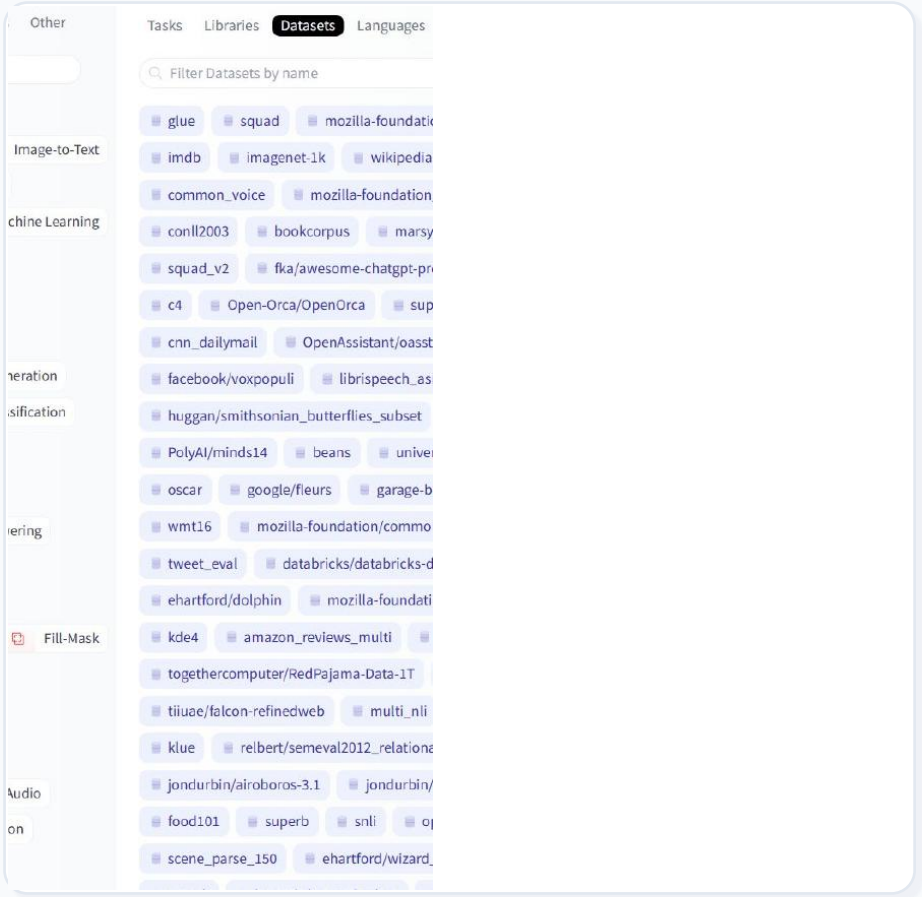
Datasets are training and evaluation data. Always read the dataset card.

Beginner workflow:

- Search a dataset by task or domain
- Open dataset card and inspect columns
- Load small split first
- Check license and data quality

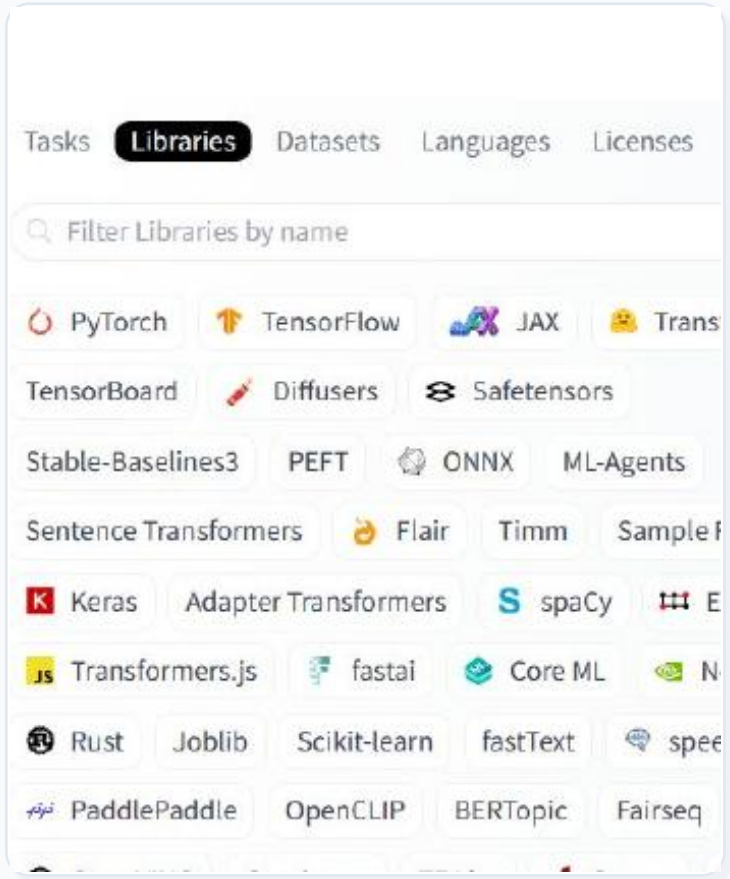
```
from datasets import load_dataset

dataset = load_dataset("imdb", split="train[:100]")
print(dataset[0])
```

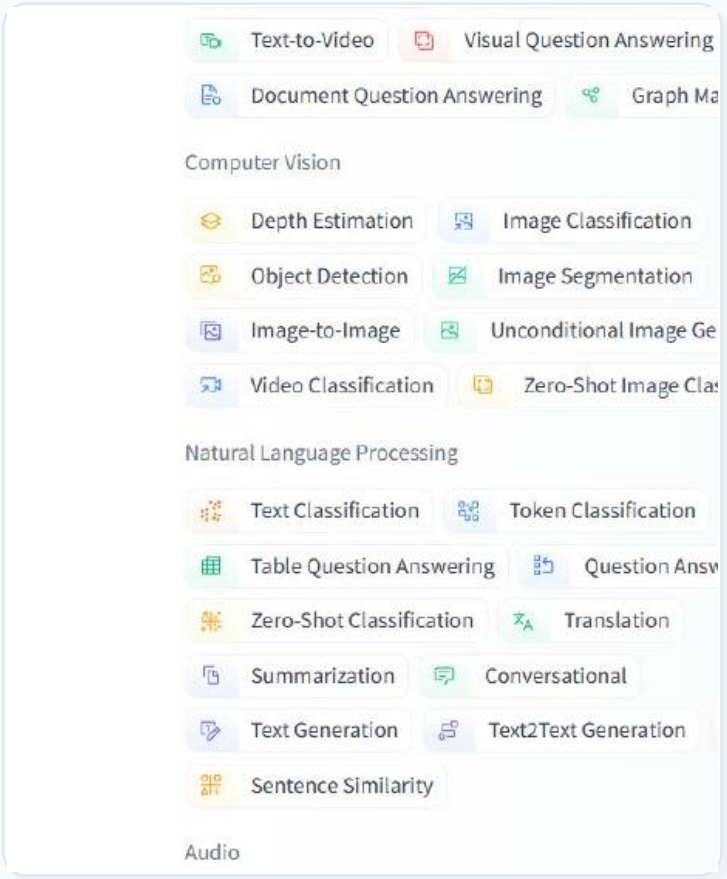


Hub filters: tasks, libraries, datasets

The Hub search page is a navigation tool. It helps you avoid random guessing.



Libraries



Tasks

Datasets

Spaces: try ML apps in the browser

Spaces are useful for demos, prototypes, and course projects.

Deploy demos in minutes

Gradio, Streamlit, Docker, and static apps are common options.

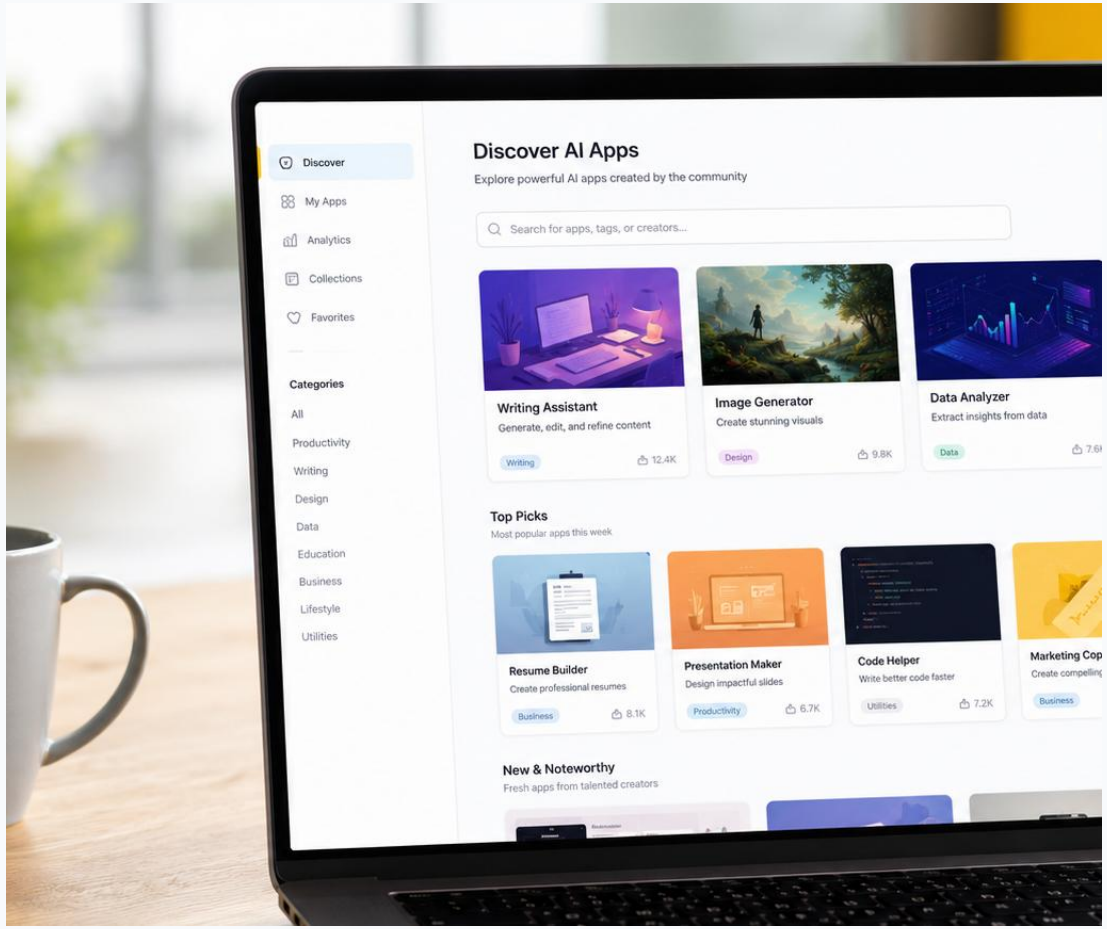
Test models interactively

Students can try inputs without writing backend code first.

Share project links

Perfect for portfolio demos and tutorials.

Use Spaces when you want students to see the model working visually.



Hands-on 2: speech recognition pipeline

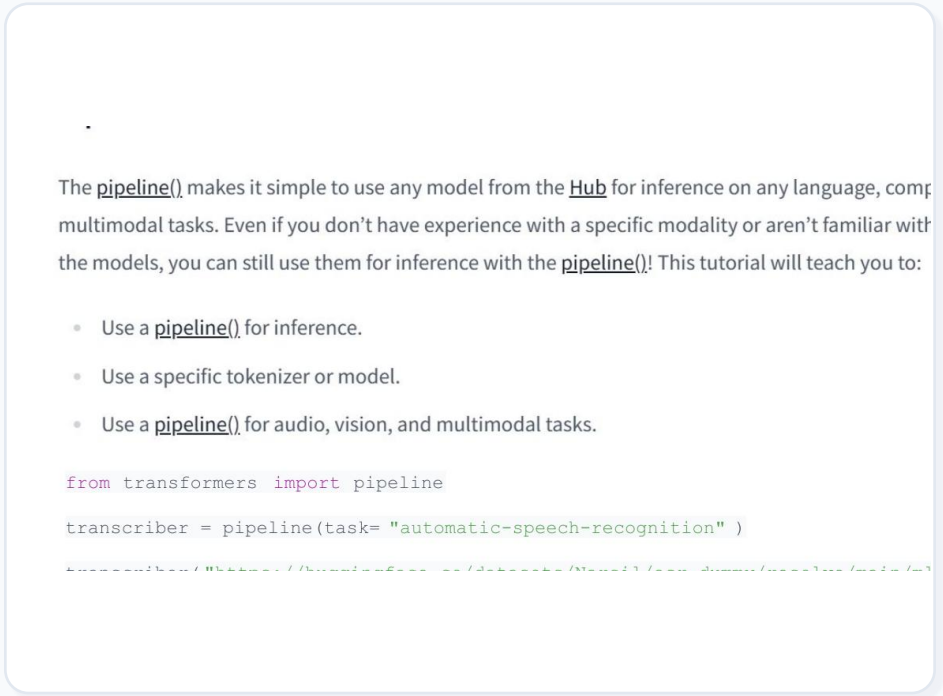
This example turns an audio file into text using a pretrained ASR model.

```
from transformers import pipeline

transcriber = pipeline(
    task="automatic-speech-recognition"
)

url =
"https://huggingface.co/datasets/Narsil/asr_dummy/resolve/mai
n/mlk.flac"

print(transcriber(url))
```



The pipeline() makes it simple to use any model from the Hub for inference on any language, comp
multimodal tasks. Even if you don't have experience with a specific modality or aren't familiar with
the models, you can still use them for inference with the pipeline()! This tutorial will teach you to:

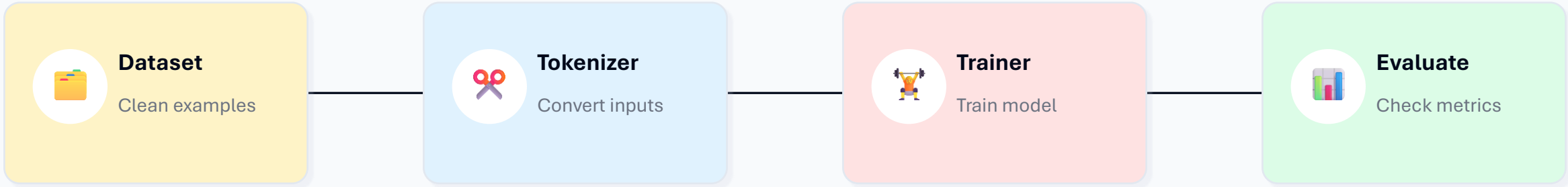
- Use a pipeline() for inference.
- Use a specific tokenizer or model.
- Use a pipeline() for audio, vision, and multimodal tasks.

```
from transformers import pipeline
transcriber = pipeline(task= "automatic-speech-recognition" )
```

Same pattern: task name + input → readable output.

Fine-tuning: when pretrained is not enough

Fine-tuning adapts a pretrained model to your own dataset and task.



Do not start with fine-tuning. First learn inference, checkpoint selection, and datasets. Fine-tuning is step two.

How to choose the right model

Use this checklist before using any checkpoint in a tutorial or project.

Question	What to check
Is it for my task?	Task tag, model card examples, pipeline support
Can I use it legally?	License, commercial restrictions, gated access
Will it run on my machine?	Model size, RAM/VRAM, quantized versions
Is it reliable enough?	Evaluation metrics, known limitations, recent updates
Can students reproduce it?	Simple code, downloads, stable dependencies

Best beginner default: use a small, popular, well-documented model first. Upgrade only after the workflow works.

KGP Talkie
LEARN. BUILD. TALK AI

Common beginner errors

Most issues are setup or checkpoint-choice problems, not Hugging Face itself.

ModuleNotFoundError

Install the missing library in the active environment.

401 / gated model

Login with token or choose an open model.

Out of memory

Use smaller model, CPU, quantization, or Colab/GPU.

Slow download

Model files are large; cache is reused after first download.

Wrong task

Choose checkpoint trained for the exact task.

License confusion

Read the model card before public/commercial use.

KGP Talkie
LEARN. BUILD. TALK AI

Practice roadmap

Use this as a mini-lab sequence for beginners.

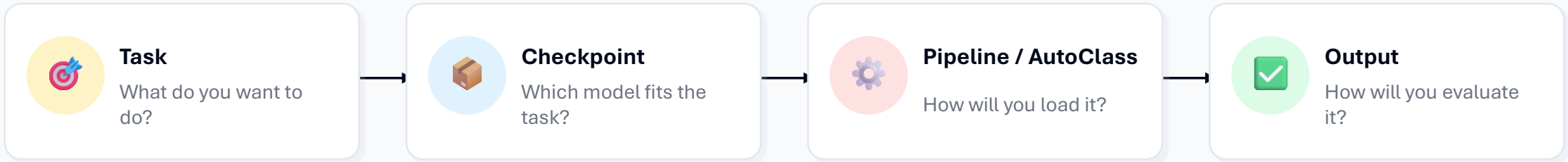
Lab 1	**Sentiment analysis pipeline**	10 minutes
Lab 2	**Question answering with context**	15 minutes
Lab 3	**Tokenizer + AutoModel**	20 minutes
Lab 4	**Load dataset and inspect rows**	15 minutes
Lab 5	**Try a Space and explain inputs/outputs**	10 minutes
Lab 6	**Mini fine-tuning explanation only**	20 minutes

Goal: by the end, students should know how to search, select, run, and explain a pretrained model.

KGP Talkie
LEARN. BUILD. TALK AI

Recap

The simplest Hugging Face learning formula.



Start with pipeline(). Grow into AutoClasses, Datasets, Spaces, and fine-tuning after the basic workflow is clear.

References used

Sources for facts, structure, and beginner examples.

Uploaded source PDF	Hugging Face Introduction.pdf
Hugging Face homepage	https://huggingface.co/
Hugging Face Hub documentation	https://huggingface.co/docs/hub/en/index
Transformers pipeline tutorial	https://huggingface.co/docs/transformers/en/pipeline_tutorial
Transformers pipelines reference	https://huggingface.co/docs/transformers/en/main_classes/pipelines
Model cards documentation	https://huggingface.co/docs/hub/en/model-cards
Spaces overview	https://huggingface.co/docs/hub/en/spaces-overview

Note: model/dataset counts change frequently. Treat public Hub numbers as current-context examples, not fixed course facts.